

Tutorial - Semana 2 - Encontro 1

Introdução

Na Semana 1 você criou o projeto do Vertical Slice e construiu seu primeiro Actor com Components — a unidade universal de composição de qualquer engine. Esta semana dá um passo além: em vez de um Actor genérico, você vai construir o `BP_Player`, o personagem controlável que vai concentrar, ao longo de todo o semestre, a locomoção, a câmera, o input e os Components de gameplay do jogador. Este encontro responde a uma pergunta específica: como uma engine resolve a locomoção de um personagem sem que você precise reimplementar física de caminhada, salto e colisão do zero?

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Distinguir Pawn de Character e justificar por que Character é uma especialização de Actor voltada a personagens controláveis.
- Explicar o papel do Character Movement Component como sistema universal de locomoção.
- Configurar um Character controlável no nível de teste do Vertical Slice.

Resultado Esperado ao Final da Semana

Um `BP_Player` funcional como subclasse de Character, posicionado no nível de teste e capaz de se mover com o controle padrão do template, com parâmetros de movimento ajustados de forma perceptível. Este é o resultado esperado ao final deste Encontro 1 especificamente — o Encontro 2 substitui o controle padrão por um esquema próprio de Enhanced Input.

Pré-requisitos

- Ter concluído a Semana 1: projeto criado, estrutura de pastas organizada e o Actor `BP_Testecomposicao` funcional.
 - Compreender o conceito de composição sobre herança e a relação entre Actor e Component.
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto `TemploEsquecido` (ou nome equivalente) da Semana 1, aberto no Unreal Editor 5.6, com a estrutura de pastas organizada.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Formas geométricas e o esqueleto/malha padrão do template Third Person, já disponíveis no projeto. O Kenney Prototype Kit só entra a partir da Semana 3.

Projeto esperado

O mesmo projeto da Semana 1, com a pasta `Blueprints/Characters` pronta para receber o `BP_Player`, e o nível `Map_Exploration` disponível para servir de nível de teste desta semana.

Parte 1 — Pawn, Character e Character Movement Component

Objetivo

Compreender por que um personagem controlável exige uma especialização própria de Actor, em vez de reaproveitar o Actor genérico construído na Semana 1.

Conceito

O `BP_TesteComposicao` da semana passada é um Actor qualquer: ele existe no mundo, mas não pode ser controlado por um jogador. Para que um Actor seja controlável, a Unreal define duas especializações em camadas:

Um **Pawn** é a classe base de qualquer Actor que pode ser "possuído" por um Controller — ou seja, que pode receber input do jogador ou de uma IA. Um Pawn puro não vem com nenhuma solução pronta de movimentação; ele só define a possibilidade de ser controlado.

Um **Character** é uma especialização de Pawn voltada especificamente a personagens bípedes ou com locomoção física complexa. A diferença essencial é que o Character já vem embutido com um **Character Movement Component**: um sistema completo que resolve caminhada, corrida, salto, natação, voo e a resolução de colisão com o chão, rampas e degraus — sem que você precise reimplementar nenhuma dessas físicas manualmente.

Esse é exatamente o mesmo problema que qualquer engine de terceira pessoa precisa resolver — apenas o grau de solução pronta entregue por padrão varia entre motores. Compreender essa separação (Actor → Pawn → Character, e o Component que resolve a locomoção) prepara você para reconhecer, na Semana 4, que GameMode e PlayerController lidam com regras e controle em um nível acima do Character.

Passo a passo

1. No Content Browser, abrir a pasta `Blueprints/Characters`.
2. Clicar com o botão direito e selecionar "Blueprint Class".
3. Na janela de seleção de classe-pai, escolher "Character" (não "Actor" nem "Pawn" — Character já embute o Character Movement Component que vamos configurar).
4. Nomear o Blueprint como `BP_Player`, seguindo a convenção `BP_` do projeto (PROJECT_ARCHITECTURE.md, seção 9).
5. Abrir o `BP_Player` com um duplo clique.
6. No painel de Components, observar que o Blueprint já vem com um Capsule Component (colisão), um Arrow Component, um Mesh Component (Skeletal Mesh) e, embutido na própria classe, um Character Movement Component — este último não aparece como um Component anexável manualmente, pois já faz parte da classe Character.
7. Selecionar o Character Movement Component na lista de Components e observar, no painel Details, as categorias de propriedades relacionadas a caminhada, salto e queda.

Resultado esperado

O editor de Blueprint mostra um Character vazio, com Capsule Component, Mesh Component e Character Movement Component já presentes por padrão, sem que você tenha precisado montá-los

manualmente como fez com o Actor puro na Semana 1.

Verificando

Confirme, no painel de Components, que o `BP_Player` tem uma hierarquia diferente do `BP_Testecomposicao`: além do Capsule e do Mesh, existe a entrada do Character Movement Component ao selecionar a raiz do Blueprint ou o próprio ícone da classe.

Problemas comuns

- **Criar como Actor ou Pawn em vez de Character:** se você escolher Actor, não haverá Character Movement Component algum; se escolher Pawn, você teria que implementar a física de locomoção manualmente. Para este exercício, a classe-pai correta é Character.
- **Tentar adicionar um "Character Movement Component" manualmente pela lista de Add Component:** ele já existe por herança da classe Character; não deve ser duplicado.

Boas práticas

Reserve a criação de Actors puros (como na Semana 1) para objetos do mundo sem necessidade de controle direto do jogador. Sempre que um Blueprint precisar ser controlável e ter locomoção física, parta de Character em vez de tentar recriar esse comportamento a partir de um Actor genérico.

Comparação com Unity

O par Pawn/Character da Unreal não tem equivalente direto único na Unity: o mais próximo é a combinação de um GameObject com CharacterController ou Rigidbody mais um script de movimento próprio. A diferença arquitetural mais relevante é que a Unreal já entrega, por padrão, um Character Movement Component completo (caminhada, corrida, salto, natação, voo, resolução de colisão com rampas e degraus), enquanto a Unity normalmente exige compor essa solução a partir de peças mais genéricas (Rigidbody, CharacterController ou pacotes de terceiros).

Parte 2 — Configurando o BP_Player no nível de teste

Objetivo

Ajustar parâmetros básicos de movimento do Character Movement Component e posicionar o `BP_Player` como o personagem controlável do nível de teste.

Conceito

Ter um Character Movement Component embutido não significa que os valores padrão sejam adequados a qualquer projeto — a engine entrega uma solução completa, mas parametrizável. Ajustar velocidade de caminhada, altura de salto e outros parâmetros é o que torna a locomoção do `BP_Player` perceptivelmente própria do Vertical Slice, em vez de idêntica ao personagem de exemplo do template.

Além disso, o `BP_Player` precisa ser reconhecido pelo nível como o Pawn que o jogador vai controlar ao iniciar a partida — isso é feito trocando a referência de Default Pawn Class nas configurações do modo de jogo padrão do template, que será formalizada com um `BP_GameMode` próprio apenas na Semana 4.

Passo a passo

1. No `BP_Player` aberto, selecionar o Character Movement Component no painel de Components.
2. No painel Details, localizar a categoria "Character Movement: Walking" e ajustar o valor de "Max Walk Speed" para um valor perceptivelmente diferente do padrão (por exemplo, aumentar ou diminuir em pelo menos 20%).
3. Localizar a categoria "Character Movement: Jumping / Falling" e ajustar o valor de "Jump Z Velocity" de forma igualmente perceptível.
4. Compilar o Blueprint (botão "Compile") e salvar.
5. Abrir o nível `Map_Exploration` (criado na Semana 1).
6. Nas configurações do projeto (Edit > Project Settings > Maps & Modes), localizar "Default Pawn Class" e trocar a referência do Pawn padrão do template para `BP_Player`.
7. Testar em modo Play (Play in Editor) e observar a movimentação no nível de teste.
8. Posicionar manualmente um Player Start no nível, caso ainda não exista um, para garantir um ponto de spawn consistente para o `BP_Player`.

Resultado esperado

Ao pressionar Play, o jogador controla o `BP_Player` no nível de teste, com velocidade de caminhada e altura de salto perceptivelmente diferentes dos valores padrão do template, usando ainda o esquema de controle padrão do projeto (teclas WASD e mouse, sem Enhanced Input customizado).

Verificando

Compare a movimentação do `BP_Player` com a memória da movimentação do personagem de exemplo do template na Semana 1: a diferença de velocidade e altura de salto deve ser claramente perceptível, não apenas numérica.

Problemas comuns

- **Nenhuma mudança perceptível na movimentação:** verifique se o Blueprint foi compilado e salvo após o ajuste dos parâmetros, e se o "Default Pawn Class" do projeto realmente aponta para `BP_Player`, não para o Pawn original do template.
- **Personagem cai pelo chão ou fica preso no nível:** confirme que existe um Player Start posicionado sobre uma superfície sólida do `Map_Exploration`.
- **Personagem não aparece no Play:** revise se o `BP_Player` foi de fato salvo como subclasse de Character (não Actor) e se o Default Pawn Class foi salvo corretamente em Project Settings.

Boas práticas

Ajuste parâmetros de movimento em pequenos incrementos, testando a cada mudança, em vez de alterar vários valores de uma vez — isso facilita identificar qual parâmetro específico produziu qual efeito, hábito importante para o Profiling que será tratado na Semana 13.

Comparação com Unity

Ajustar "Max Walk Speed" e "Jump Z Velocity" no Character Movement Component da Unreal é equivalente a ajustar parâmetros de velocidade e força de salto em um script de movimento próprio sobre um CharacterController da Unity. A diferença é que, na Unreal, esses parâmetros já existem prontos como propriedades expostas no painel Details, enquanto na Unity eles normalmente são variáveis que o próprio time precisa declarar e programar dentro do script de movimento.

Ao final da semana

Este Encontro 1 cobre a primeira metade da Semana 2. Ao final dele, o projeto deve conter um `BP_Player` funcional como subclasse de `Character`, posicionado no nível de teste (`Map_Exploration`) e controlável com o esquema padrão do template, com parâmetros de movimento ajustados de forma perceptível. Isso corresponde ao início da linha "BP_Player (locomção)" do roadmap do Módulo 1 no `PROJECT_ARCHITECTURE.md`. O controle padrão do template ainda está em uso — ele será substituído por um esquema próprio de Enhanced Input no Encontro 2.

Desafio

Não há desafio neste encontro — a configuração do `BP_Player` é demonstração e replicação guiada, coerente com a autonomia muito baixa do Módulo 1 (`PEDAGOGICAL_RULES.txt`). O primeiro desafio da semana ocorre no Encontro 2.

Checklist

- ☐ `BP_Player` criado como subclasse de `Character` (não `Actor` nem `Pawn`), na pasta `Blueprints/Characters`
- ☐ Consigo explicar de memória a diferença entre `Pawn` e `Character`
- ☐ "Max Walk Speed" e "Jump Z Velocity" ajustados para valores perceptivelmente diferentes do padrão
- ☐ "Default Pawn Class" do projeto apontando para `BP_Player`
- ☐ `BP_Player` controlável no `Map_Exploration` em modo Play, com Player Start posicionado
- ☐ Blueprint compilado sem erros e salvo

Glossário

- **Pawn:** classe base de qualquer `Actor` que pode ser possuído por um `Controller` (jogador ou IA), sem solução própria de locomoção.
- **Character:** especialização de `Pawn` voltada a personagens com locomoção física complexa, já embutindo um `Character Movement Component`.
- **Character Movement Component:** sistema universal de locomoção embutido no `Character`, responsável por caminhada, corrida, salto, natação, voo e resolução de colisão com o terreno.

- **Player Start:** Actor que define o ponto de spawn do Pawn controlado pelo jogador ao iniciar a partida.
- **Default Pawn Class:** configuração do projeto que define qual classe de Pawn é automaticamente possuída pelo jogador ao iniciar o nível.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine (Character e Pawn). Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Character Movement. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — CharacterController e Rigidbody, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Character Movement Component.